# L'Architecture Client-Serveur et les Fondements du Protocole HTTP

**Thème principal :** Fonctionnement du Web, Cycle Requête/Réponse et Codes de Statut HTTP

**Synthèse exécutive :** Ce cours pose les bases indispensables du développement d'interfaces web en détaillant l'architecture client-serveur et les mécanismes sous-jacents du protocole HTTP (Hypertext Transfer Protocol). L'enjeu est de comprendre comment un navigateur (le client) formule une demande normalisée à un serveur hébergeant des ressources, comment ce dernier traite la requête et renvoie une réponse structurée (en-têtes, code de statut, corps), et pourquoi le caractère 'sans état' (stateless) d'HTTP impose des mécanismes complémentaires comme les cookies ou les sessions.

## 1. Le Modèle Client-Serveur dans l'Écosystème Web

Dans l'architecture web, le client et le serveur ont des rôles asymétriques stricts. Le client prend toujours l'initiative de la connexion. Pour afficher une page web, le navigateur commence par résoudre le nom de domaine en adresse IP via le protocole DNS, ouvre un socket TCP avec le serveur distant sur le port dédié (généralement 80 ou 443), puis transmet un message formaté selon la spécification HTTP. Le serveur reçoit ce message, identifie la ressource demandée, applique d'éventuels traitements côté serveur (scripts PHP, requêtes SQL, exécution de logique applicative) et restitue un flux binaire ou textuel accompagné de métadonnées décrivant la ressource.

- Client (User-Agent) : entité initiatrice de la communication (navigateur, client cURL, application mobile)
- Serveur : machine hôte en écoute passive sur un port réseau (port 80 pour HTTP, 443 pour HTTPS) traitant les requêtes
- Protocole applicatif : ensemble de règles formelles régissant la syntaxe et la sémantique des échanges au-dessus de TCP/IP

***Cas concret enseignant :*** *L'analogie du restaurant : Le client est le convive assis à la table. Le serveur web est le serveur de salle qui attend la commande. Le client passe une commande formelle (la requête) à partir d'une carte (l'URL/URI). Le serveur transmet la commande en cuisine (la base de données ou le backend) et revient avec l'assiette (la ressource HTML demandée) accompagnée d'un ticket confirmant la commande (l'en-tête de réponse).*

## 2. Anatomie d'une Requête et d'une Réponse HTTP

Une requête HTTP commence par une ligne de requête obligatoire (ex: 'GET /cours.html HTTP/1.1'), suivie d'une série d'en-têtes HTTP terminés par une ligne vide (séquence CRLF : retour chariot et saut de ligne), puis éventuellement d'un corps de message. La méthode 'GET' est employée pour récupérer des données sans effet de bord sur le serveur (idempotente et sûre). La méthode 'POST' transmet des données dans le corps de la requête, généralement pour modifier l'état du serveur ou enregistrer des informations sensibles (formulaire d'inscription, identifiants). La réponse suit la même logique structurelle : la première ligne contient le code de statut (ex: 'HTTP/1.1 200 OK'), suivie des en-têtes (ex: 'Content-Type: text/html; charset=UTF-8', 'Content-Length: 1024'), puis d'une ligne vide séparatrice et du code HTML brut interprété par le moteur de rendu du navigateur.

- Ligne de commande (Request Line) : Méthode HTTP (GET, POST, PUT, DELETE), URI (/index.html) et Version du protocole (HTTP/1.1)
- En-têtes (Headers) : paires clé-valeur fournissant le contexte (Host, User-Agent, Accept, Content-Type)
- Ligne de statut (Status Line) : Version du protocole, Code numérique (ex: 200, 404) et Message associé (Reason Phrase)
- Corps (Body / Payload) : contenu effectif transmis (données de formulaire en POST, contenu HTML/JSON en réponse)

***Cas concret enseignant :*** *Démonstration dans les Outils de Développement du navigateur (touche F12, onglet 'Réseau' ou 'Network') : observation en direct de chaque requête générée lors de l'actualisation d'une page, montrant la séparation nette entre le premier appel du document HTML et les dizaines de sous-requêtes ultérieures pour charger les fichiers CSS, images et scripts JavaScript.*

## 3. Typologie des Codes de Statut HTTP (2xx, 3xx, 4xx, 5xx)

Les codes de statut HTTP sont des entiers à 3 chiffres standardisés par l'IETF. Le premier chiffre catégorise la nature de la réponse. La maîtrise de ces codes est fondamentale pour tout développeur car ils conditionnent la gestion des erreurs dans l'interface et la communication avec les API Web. Une erreur 404 signifie que l'URI demandée n'existe pas sur le serveur (faute de frappe dans l'URL ou fichier déplacé sans redirection). Une erreur 500 indique qu'une exception non gérée s'est produite lors de l'exécution du code serveur (ex: erreur de syntaxe PHP ou panne de connexion à la base de données). Une redirection 304 signale au navigateur que sa version en cache local est toujours à jour, évitant ainsi un re-téléchargement inutile.

- Famille 2xx (Succès) : la requête a été reçue, comprise et acceptée (ex: 200 OK, 201 Created)
- Famille 3xx (Redirection) : une action supplémentaire est requise pour obtenir la ressource (ex: 301 Moved Permanently, 302 Found, 304 Not Modified)
- Famille 4xx (Erreur Client) : anomalie côté émetteur (ex: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found)
- Famille 5xx (Erreur Serveur) : le serveur a échoué à répondre à une requête apparemment valide (ex: 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable)

***Cas concret enseignant :*** *La règle mnémotechnique en évaluation : Si le problème vient de l'URL tapée par l'internaute ou de son absence de connexion/droits, c'est du 4xx (responsabilité client). Si le script PHP plante ou que la base MariaDB est éteinte, c'est du 5xx (responsabilité serveur). Ne confondez jamais une page introuvable (404) avec un crash d'application (500).*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi ne doit-on jamais envoyer un mot de passe avec la méthode GET dans un formulaire HTML ?
**Réponse du professeur :** Avec la méthode GET, l'ensemble des données saisies dans les champs de formulaire est encodé directement dans l'URL sous la forme d'une chaîne de requête (Query String, ex: /login?user=toto&pass;=secret123). Cette URL est conservée en clair dans l'historique du navigateur, apparaît sur l'écran aux yeux de tous, est stockée dans les journaux d'accès (logs) des serveurs web et proxys, et peut fuiter via l'en-tête 'Referer'. On utilise impérativement la méthode POST, qui transporte les données dans le corps (body) de la requête HTTP, idéalement chiffré via HTTPS (TLS).

**Question étudiant :** Si le protocole HTTP est sans état (stateless), comment un site de e-commerce se souvient-il des articles dans mon panier quand je change de page ?
**Réponse du professeur :** HTTP est effectivement sans mémoire native : chaque requête est indépendante de la précédente. Pour créer l'illusion d'une session continue, le serveur envoie lors de la première interaction un en-tête 'Set-Cookie' contenant un identifiant unique de session (Session ID). Le navigateur stocke ce cookie et le renvoie automatiquement au serveur dans l'en-tête 'Cookie' à chaque requête ultérieure. Le serveur utilise cet identifiant pour retrouver les données associées en mémoire ou en base de données côté serveur.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Analysez la trame brute suivante interceptée lors d'un échange HTTP et déduisez-en : 1) Le rôle de la trame (requête ou réponse), 2) La ressource demandée et la méthode utilisée, 3) Le type MIME du contenu et l'interprétation du code de retour. Trame interceptée : HTTP/1.1 200 OK Date: Mon, 19 Oct 2026 14:22:00 GMT Server: Apache/2.4.52 (Ubuntu) Content-Type: text/html; charset=UTF-8 Content-Length: 154 Accueil Bienvenue en R1.02

**Résolution pas à pas :**
Étape 1 : Identification de la nature de la trame. La première ligne commence par la version du protocole suivie d'un code numérique et d'un message textuel ('HTTP/1.1 200 OK'). Il s'agit donc sans ambiguïté d'une ligne de statut (Status Line) caractéristique d'une RÉPONSE HTTP émise par un serveur vers un client. Étape 2 : Analyse de la ressource et de la méthode. La trame étant une réponse, elle ne contient pas directement la ligne de requête (qui figurait dans le message émis au préalable par le client). Toutefois, les en-têtes et le corps indiquent que le serveur délivre la ressource finale. Étape 3 : Analyse des métadonnées et du code de statut. - Le code '200 OK' atteste que la requête précédente a été traitée avec succès. - L'en-tête 'Content-Type: text/html; charset=UTF-8' informe le navigateur que le corps de la réponse doit être interprété comme un document balisé HTML avec l'encodage de caractères UTF-8. - L'en-tête 'Content-Length: 154' indique que le corps fait exactement 154 octets. - Après la ligne vide obligatoire, le corps contient le balisage HTML prêt à être interprété par le moteur de rendu.

**Résultat : Trame validée : il s'agit d'une réponse HTTP/1.1 valide délivrant un document HTML simple (154 octets) avec succès (code 200 OK) depuis un serveur web Apache.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence fondamentale entre les codes HTTP 401 Unauthorized et 403 Forbidden ?**

*R : Le code 401 indique que l'accès requiert une authentification préalable (l'utilisateur n'est pas identifié ou ses identifiants sont invalides). Le code 403 signifie que le serveur a bien identifié le client, mais refuse formellement de lui donner accès à la ressource car ses privilèges/droits sont insuffisants.*

**Q : Citez les quatre composants obligatoires formant la première ligne d'une requête HTTP (Request Line).**

*R : La méthode HTTP (ex: GET, POST), un espace, l'URI relative de la ressource demandée (ex: /images/logo.png), un espace, et la version du protocole (ex: HTTP/1.1).*

**Q : Que signifie la notion d'idempotence pour une méthode HTTP comme GET ou PUT ?**

*R : Une méthode est idempotente si l'exécuter plusieurs fois de manière identique produit exactement le même effet sur l'état du système/serveur qu'une seule exécution. GET est idempotente (la lecture répétée ne modifie rien), tandis que POST ne l'est généralement pas (plusieurs soumissions peuvent créer plusieurs enregistrements).*